SMARTBRIDGE

SKILL WALL • Coding & Solution

Coding & Solution

Date	01 October 2026
Team ID	SWTID-2026-2430
Project Name	Team Enrollment Confirmation - FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Coding & Solution

This section describes the FitBuddy implementation. The project uses FastAPI with Google Gemini models to generate personalized 7-day workout plans, nutrition or recovery tips, and feedback-based plan updates. SQLite with SQLAlchemy stores user and plan information, while HTML/Jinja2 templates provide the interactive interface.

Instructions:

1. Ensure the code is modular, structured, documented, and follows good practices for the selected technology stack.

2. The solution addresses the core FitBuddy problem statement and implements workout generation, nutrition tips, feedback-based updates, data storage, and an admin view.

3. Include source files, configuration files, requirements, environment variables, and setup instructions needed to run the application.

4. Add the GitHub/GitLab repository URL when it is available.

Solution Summary

Field	Details
Repository Link / URL	To be added by the team
Programming Language(s)	Python, HTML, CSS
Framework(s) Used	FastAPI, Jinja2, SQLAlchemy
AI Models	Gemini 1.5 Pro for workout generation and feedback updates; Gemini Flash for nutrition/recovery tips
Key Features Implemented	Personalized 7-day workout plan; nutrition/recovery tip; feedback-based plan update; SQLite storage; admin user/plan view; API endpoints; responsive frontend
Pending / Incomplete Features	Repository URL and final deployment details, if not yet added by the team
Setup / Run Instructions	Create virtual environment → install dependencies → configure GOOGLE_API_KEY → run Uvicorn → open http://127.0.0.1:8000 and /docs for API testing

Team Work Allocation

The work is assigned to the five members in the exact order of names provided, covering the five project milestones documented in the FitBuddy project material.

S.No	Team Member	Assigned Area	Responsibilities
1	Divya.D	Milestone 1 – Model Selection & Architecture	Research and select Gemini models; define the application architecture; set up the development environment; document the FastAPI, Gemini, database and frontend integration.
2	Deepika.M	Milestone 2 – Core Functionalities	Develop workout-plan generation and nutrition-tip functions; integrate Gemini 1.5 Pro and Gemini Flash; implement user and plan storage using SQLite and SQLAlchemy.
3	Dharani.T	Milestone 3 – routes.py / Backend Logic	Develop routes.py; implement home, generate-workout, submit-feedback and view-all-users routes; process form input; connect AI and database functions.
4	Divya shree.R	Milestone 4 – Frontend Development	Design index.html, result.html and all_users.html; develop responsive CSS; integrate Jinja2 templates; bind backend data to the frontend.
5	DivyaDharshini.V	Milestone 5 – Deployment, Testing & Verification	Prepare local deployment; configure virtual environment, requirements and GOOGLE_API_KEY; run Uvicorn; test major user flows and API documentation; record final verification.

Code Quality Checklist

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'S.No' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='DV', fontSize=8.6, greek=0, italic=1, link=[], rise=0, text='S.No', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Criteria' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='DV', fontSize=8.6, greek=0, italic=1, link=[], rise=0, text='Criteria', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Status (Yes / No)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='DV', fontSize=8.6, greek=0, italic=1, link=[], rise=0, text='Status (Yes / No)', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	To be confirmed

Additional Notes / Comments

Team members should coordinate across milestones so the AI layer, backend routes, frontend templates, database, deployment and testing work together.
The project material identifies modules/files including gemini_generator.py, gemini_flash_generator.py, updated_plan.py, database.py, routes.py and the Jinja2 templates.

The repository URL can be inserted after the team creates or confirms the GitHub/GitLab repository.